



GURU GOBIND SINGH FOUNDATION'S
GURU GOBIND SINGH COLLEGE OF ENGINEERING AND RESEARCH CENTRE, NASHIK

Certificate

This is to certify that Mr./Miss _____ of Third Year Artificial Intelligence and Data Science, Roll No.:____ University Examination, Seat Number_____ has completed all the practical work/Term work in Subject_____ satisfactorily in **Department of Artificial Intelligence and Data Science** as prescribed by the **Savitribai Phule Pune University** in the academic year 20__ - 20__.

Practical In-charge

Head of the Department

Principal

Index

Practical No.	Name of Experiment	Date of Performance	Date of Completion	Grade/ Marks	Sign
1	Data Wrangling-I: Importing libraries, loading a dataset, preprocessing it, checking for missing values, exploring the dataset, and performing necessary transformations and normalizations to clean the data.				
2	Data Wrangling-II: Create a synthetic "Academic Performance" dataset for students, and then perform various data wrangling operations in Python. The tasks will involve: 1. Scanning the dataset for missing values and inconsistencies, and handling them. 2. Identifying and dealing with outliers in numeric variables. 3. Applying data transformations on at least one variable for better understanding or to improve distribution.				
3	Descriptive Statistics - Measures of Central Tendency and variability: Using the Iris dataset. We will: a) Perform operations on the Iris dataset to calculate basic statistical details (percentile, mean, standard deviation, etc.) for different species ('Iris-setosa', 'Iris-versicolor', 'Iris-virginica'). b) Use Python to extract and analyze these statistics for each species and understand their distributions.				
4	Data Analytics-I: Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset. The Boston Housing dataset contains information about various houses in Boston through different parameters. There are 506 samples and 14 feature variables in this dataset. The objective is to predict the value of prices of the house using the given features				

5	Data Analytics-II 1. Implement logistic regression using Python/R to perform classification on Social_Network_Ads.csv dataset. 2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.				
6	Data Analytics-III 1. Implement Simple Naïve Bayes classification algorithm using Python/R on iris.csv dataset. 2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.				
7	Text Analytics 1. Extract Sample document and apply following document preprocessing methods: Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization. 2. Create representation of documents by calculating Term Frequency and Inverse DocumentFrequency.				
8	Data Visualization I: 1. Use the inbuilt dataset 'titanic'. The dataset contains 891 rows and contains information about the passengers who boarded the unfortunate Titanic ship. Use the Seaborn library to see if we can find any patterns in the data. 2. Write a code to check how the price of the ticket (column name: 'fare') for each passenger is distributed by plotting a histogram.				
9	Data Visualization II: On Titanic dataset 1. Plot a box plot for distribution of age with respect to each gender along with the information about whether they survived or not. (Column names : 'sex' and 'age') 2. Write observations on the inference from the above statistics.				
10	Data Visualization III: On iris dataset give inference as: 1. List down the features and their types (e.g., numeric, nominal) available in the dataset.				

	2. Create a histogram for each feature in the dataset to illustrate the feature distributions. 3. Create a boxplot for each feature in the dataset. 4. Compare distributions and identify outliers				
11	Create databases and tables, insert small amounts of data, and run simple queries using Impala				
12	Write a simple program in SCALA using Apache Spark framework				
13	Mini Project				

Date of Submission:

Practical In-charge

Data Wrangling-I: Preprocessing and Normalizing Dataset in Python

Objective

The objective of this document is to guide you through the data wrangling process using Python on any open-source dataset. This will include importing libraries, loading a dataset, preprocessing it, checking for missing values, exploring the dataset, and performing necessary transformations and normalizations to clean the data.

Hardware Requirements

- **Processor:** Any modern processor (Intel i5 or higher recommended)
 - **RAM:** 4GB or more (8GB or higher recommended)
 - **Storage:** 10GB or more free space (for storing datasets and Python environment)
-

Software Requirements

- **Python (version 3.x):** Python programming language, commonly used in data science and machine learning.
 - **Anaconda:** A distribution that comes with Python, Jupyter Notebooks, and all necessary packages for data science.
 - **Libraries/Modules:**
 - `pandas` for data manipulation
 - `numpy` for numerical operations
 - `matplotlib` and `seaborn` for visualization
 - `scikit-learn` for machine learning (if needed)
 - `requests` and `beautifulsoup4` for web scraping (if needed)
-

Theory

Data wrangling (also known as data cleaning or data preprocessing) is the process of transforming and mapping raw data into a more usable format for analysis. It involves operations such as handling missing values, formatting data types, removing duplicates, normalizing data, and transforming categorical variables into numerical variables.

The major steps of data wrangling in Python are:

1. **Importing necessary libraries.**
2. **Locating and importing data.**
3. **Exploring and preprocessing the data.**
4. **Checking and handling missing data.**
5. **Data transformation and normalization.**
6. **Converting categorical variables into numerical formats.**

These operations help prepare the dataset for effective analysis, machine learning, or statistical modeling.

Steps

1. Import all the required Python Libraries

To start, we need to import the essential libraries for data manipulation, analysis, and visualization.

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

These libraries allow us to load data, perform analysis, and visualize the dataset effectively.

2. Locate open-source data from the web

For this step, we will locate an open-source dataset from Kaggle (a popular platform for datasets). For example, let's use the **Titanic** dataset available on Kaggle.

- **Dataset Description:** The Titanic dataset contains information about passengers who were aboard the Titanic ship. It includes variables like age, sex, class, and whether the passenger survived or not.
 - **Data Source URL:** [Titanic dataset on Kaggle](#)
-

3. Load the Dataset into Pandas DataFrame

We can use `pandas` to load the dataset into a DataFrame for manipulation. If you download the dataset manually, you can use the following code:

```
# Load the dataset into pandas dataframe
df = pd.read_csv('titanic.csv') # Change the file path to your dataset's path
```

The `read_csv()` function loads a CSV file into a DataFrame where rows represent observations and columns represent variables.

4. Data Preprocessing: Check for Missing Values

Next, we will explore the dataset by checking for missing values and getting some initial descriptive statistics. This will help us understand the structure and cleanliness of the dataset.

```
# Check for missing values
missing_values = df.isnull().sum()
print("Missing Values in each column:\n", missing_values)
```

```
# Get basic statistics about the data
data_description = df.describe()
print("Initial Statistics:\n", data_description)
```

```
# Check the data types of variables
print("Data Types:\n", df.dtypes)
```

```
# Check dimensions of the DataFrame
print("Shape of the data:", df.shape)
```

Explanation:

- `isnull().sum()` tells you how many missing values exist in each column.
 - `describe()` gives you summary statistics such as count, mean, and standard deviation.
 - `dtypes` returns the data types of each column.
 - `shape` provides the number of rows and columns in the DataFrame.
-

5. Data Formatting and Data Normalization

In this step, we will inspect the types of variables and apply necessary type conversions to ensure they are in the correct format. We may also normalize or scale the numeric features if needed.

```
# Check data types of columns
print(df.dtypes)

# Convert 'Age' column to numeric if not already
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')

# Convert 'Survived' column to categorical
df['Survived'] = df['Survived'].astype('category')

# Normalize numeric columns (e.g., 'Age', 'Fare')
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])

# Check the types after conversion
print("Data Types After Conversion:\n", df.dtypes)
```

Explanation:

- We use `pd.to_numeric()` to convert columns like age to numeric values.
 - `astype('category')` is used to convert the "Survived" column into a categorical variable.
 - `MinMaxScaler` scales numeric features to a range between 0 and 1.
-

6. Turn Categorical Variables into Quantitative Variables

Categorical variables such as "Sex" and "Embarked" can be transformed into quantitative variables (numeric form) by encoding them.

```
# Convert 'Sex' column to numerical (0 = male, 1 = female)
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Convert 'Embarked' column to numerical (using one-hot encoding)
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)
```

Explanation:

- `map()` is used to convert categorical values like 'male' and 'female' into 0 and 1, respectively.
- `pd.get_dummies()` is used for one-hot encoding categorical variables like "Embarked", converting them into multiple binary columns.

Important Coding Concepts

1. **Pandas DataFrame:** A two-dimensional labeled data structure used for data manipulation.
 2. **Missing Data Handling:** Identifying and dealing with missing values using functions like `isnull()` and `fillna()`.
 3. **Data Normalization:** Scaling data to ensure consistency and improve the performance of algorithms.
 4. **Categorical Encoding:** Converting categorical variables into numerical formats (e.g., using `map()` or `get_dummies()`).
 5. **Data Types:** Understanding and converting data types using `astype()` and `pd.to_numeric()`.
-

Conclusion

In this exercise, we performed essential data wrangling tasks such as loading a dataset, checking for missing values, summarizing statistics, normalizing numeric columns, and transforming categorical variables into quantitative formats. These operations help us prepare the dataset for further analysis or machine learning models by cleaning and standardizing the data. Data wrangling is a crucial first step in any data science workflow, as it ensures that the data is in a usable and reliable format.

Data Wrangling-II: Academic Performance Dataset

Objective

The objective of this exercise is to create a synthetic "Academic Performance" dataset for students, and then perform various data wrangling operations in Python. The tasks will involve:

4. Scanning the dataset for missing values and inconsistencies, and handling them.
 5. Identifying and dealing with outliers in numeric variables.
 6. Applying data transformations on at least one variable for better understanding or to improve distribution.
-

Hardware Requirements

- **Processor:** Any modern processor (Intel i5 or higher recommended)
 - **RAM:** 4GB or more (8GB or higher recommended)
 - **Storage:** 10GB or more free space (for storing datasets and Python environment)
-

Software Requirements

- **Python (version 3.x):** Python programming language, commonly used in data science and machine learning.
 - **Anaconda:** A distribution that includes Python, Jupyter Notebooks, and essential packages for data science.
 - **Libraries/Modules:**
 - **pandas** for data manipulation
 - **numpy** for numerical operations
 - **matplotlib** and **seaborn** for visualization
 - **scikit-learn** for handling outliers (optional)
-

Theory

Steps

1. Create an "Academic Performance" Dataset

We will start by generating a synthetic dataset representing students' academic performance. The dataset will have the following columns:

- **Student ID:** Unique identifier for each student.
- **Age:** Age of the student.
- **Gender:** Gender of the student (Male/Female).
- **Subject 1:** Score in Mathematics.
- **Subject 2:** Score in English.
- **Subject 3:** Score in Science.
- **Attendance:** Percentage of attendance.
- **Final Grade:** The final grade the student achieved (A/B/C).

```
import pandas as pd
import numpy as np
```

```
# Create a synthetic dataset for Academic Performance
```

```
data = {
    'Student ID': range(1, 101),
    'Age': np.random.randint(18, 25, 100), # Age between 18 and 25
    'Gender': np.random.choice(['Male', 'Female'], 100),
    'Subject 1': np.random.randint(50, 100, 100), # Mathematics score
    'Subject 2': np.random.randint(60, 90, 100), # English score
    'Subject 3': np.random.randint(55, 95, 100), # Science score
    'Attendance': np.random.uniform(70, 100, 100), # Attendance percentage
    'Final Grade': np.random.choice(['A', 'B', 'C'], 100) # Grade
}
```

```
# Convert to DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Introduce some missing values randomly for demonstration
```

```
df.loc[5, 'Subject 1'] = np.nan
df.loc[20, 'Subject 2'] = np.nan
df.loc[35, 'Attendance'] = np.nan
```

2. Scan for Missing Values and Inconsistencies

Now, we will scan the dataset for any missing values and handle them using suitable techniques. We will also check for inconsistencies, such as negative values in "Attendance" or out-of-range scores.

```
# Check for missing values
missing_values = df.isnull().sum()
print("Missing Values:\n", missing_values)

# Handle missing values using imputation or deletion
df['Subject 1'].fillna(df['Subject 1'].mean(), inplace=True) # Fill missing math score with mean
df['Subject 2'].fillna(df['Subject 2'].median(), inplace=True) # Fill missing English score with median
df['Attendance'].fillna(df['Attendance'].mean(), inplace=True) # Fill missing Attendance with mean

# Check for negative or inconsistent values (Attendance cannot be less than 0 or greater than 100)
df['Attendance'] = df['Attendance'].clip(0, 100)

# Ensure "Final Grade" is consistent
df['Final Grade'] = df['Final Grade'].apply(lambda x: x if x in ['A', 'B', 'C'] else 'C')

# Verify no missing values remain
missing_values_after = df.isnull().sum()
print("\nMissing Values After Imputation:\n", missing_values_after)
```

Explanation:

- Missing values in **Subject 1**, **Subject 2**, and **Attendance** are filled using imputation techniques (mean for numeric columns, median for skewed data).
- We also clip the attendance values to ensure they fall within the valid range of 0 to 100.
- Finally, we check for any remaining inconsistencies in the **Final Grade** column.

3. Scan for Outliers in Numeric Variables

Next, we will identify and handle outliers in numeric variables such as **Subject 1**, **Subject 2**, **Subject 3**, and **Attendance**. We will use the Interquartile Range (IQR) method for detecting outliers.

```
# Function to detect outliers using IQR
def detect_outliers(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
```

```
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
return df[(df[column] < lower_bound) | (df[column] > upper_bound)]
```

```
# Detect outliers for 'Subject 1'
outliers_subject_1 = detect_outliers(df, 'Subject 1')
print(f"\nOutliers in Subject 1:\n{outliers_subject_1}")

# Remove outliers by clipping them to the IQR bounds
df['Subject 1'] = np.clip(df['Subject 1'], df['Subject 1'].quantile(0.25) - 1.5 * (df['Subject
1'].quantile(0.75) - df['Subject 1'].quantile(0.25)),
                        df['Subject 1'].quantile(0.75) + 1.5 * (df['Subject 1'].quantile(0.75) - df['Subject
1'].quantile(0.25)))

# Detect outliers for 'Subject 2'
outliers_subject_2 = detect_outliers(df, 'Subject 2')
print(f"\nOutliers in Subject 2:\n{outliers_subject_2}")

# Similar outlier removal for other numeric columns can be done
```

Explanation:

- We use the IQR method to identify outliers, considering data points that lie outside 1.5 times the interquartile range as outliers.
- Outliers are handled by clipping them within the valid IQR bounds, ensuring that they no longer influence the analysis unduly.

4. Apply Data Transformations

Now, we will apply data transformation to one of the variables. For example, if **Subject 1** scores are skewed, we can apply a log transformation to normalize the distribution.

```
# Check the skewness of 'Subject 1' before transformation
print("Skewness of Subject 1 before transformation:", df['Subject 1'].skew())

# Apply log transformation to 'Subject 1' to reduce skewness
df['Subject 1'] = np.log(df['Subject 1'] + 1)

# Check skewness after transformation
print("Skewness of Subject 1 after transformation:", df['Subject 1'].skew())
```

Explanation:

- Skewness is a measure of asymmetry in the distribution of a variable. If a variable is highly skewed, applying a logarithmic transformation can help normalize it.
- We apply `np.log()` to **Subject 1**, adjusting the values to handle any skewness and improve the distribution.

Important Coding Concepts

1. **Imputation:** Replacing missing values with the mean, median, or mode of a column to maintain data integrity.
 2. **IQR (Interquartile Range):** A method for detecting outliers by analyzing the distribution of data between the 25th and 75th percentiles.
 3. **Clipping:** Limiting the values of a variable to a specified range, often used to handle outliers.
 4. **Log Transformation:** A transformation used to reduce the skewness of a variable and normalize its distribution.
 5. **Data Cleaning:** Ensuring that the dataset is free from inconsistencies and missing values before further analysis.
-

Conclusion

In this exercise, we created an "Academic Performance" dataset and performed critical data wrangling operations, including handling missing values, detecting and dealing with outliers, and applying data transformations. These operations are fundamental in preparing data for analysis or machine learning models. Data wrangling improves the quality and reliability of the dataset, leading to more accurate insights and predictions.

Descriptive Statistics: Measures of Central Tendency and Variability

Objective

The objective of this exercise is to demonstrate the application of descriptive statistics—specifically measures of central tendency and variability—using the Iris dataset. We will:

1. Perform operations on the Iris dataset to calculate basic statistical details (percentile, mean, standard deviation, etc.) for different species ('Iris-setosa', 'Iris-versicolor', 'Iris-virginica').
 2. Use Python to extract and analyze these statistics for each species and understand their distributions.
-

Hardware Requirements

- **Processor:** Any modern processor (Intel i5 or higher recommended)
 - **RAM:** 4GB or more (8GB or higher recommended)
 - **Storage:** 10GB or more free space (for storing datasets and Python environment)
-

Software Requirements

- **Python (version 3.x):** Python programming language, commonly used for data science and statistical analysis.
 - **Anaconda:** A distribution that comes with Python, Jupyter Notebooks, and all necessary libraries for data analysis.
 - **Libraries/Modules:**
 - `pandas` for data manipulation
 - `numpy` for numerical operations
 - `matplotlib` and `seaborn` for visualization (optional)
-

Theory

Descriptive Statistics

Descriptive statistics is the branch of statistics that deals with summarizing and describing the features of a dataset. There are two main types of descriptive statistics:

1. **Measures of Central Tendency:** These measures describe the center or typical value of a dataset. The most common measures are:
 - **Mean:** The average of all data points.
 - **Median:** The middle value of a dataset when arranged in order.
 - **Mode:** The value that occurs most frequently in the dataset.
2. **Measures of Variability (or Spread):** These measures describe how spread out or dispersed the data points are. Common measures include:
 - **Standard Deviation:** The average amount of deviation from the mean. A high standard deviation indicates a high variability in the data.
 - **Variance:** The square of the standard deviation, which gives the dispersion of data points.
 - **Percentiles and Quartiles:** Percentiles divide the data into 100 equal parts, while quartiles divide it into four parts. The first quartile (Q1) is the 25th percentile, the second quartile (Q2) is the median (50th percentile), and the third quartile (Q3) is the 75th percentile.

The **Iris dataset** is a widely used dataset in machine learning and statistics. It contains measurements of 150 Iris flowers from three species ('Iris-setosa', 'Iris-versicolor', 'Iris-virginica'):

- **Sepal length**
- **Sepal width**
- **Petal length**
- **Petal width**

We will calculate basic statistical measures for each species.

Steps

1. Load the Iris Dataset

We'll use the **Iris dataset** to perform the statistical operations. If you don't have the dataset, you can easily find it in CSV format online, or it may be available within the Python **seaborn** or **sklearn** libraries.

```
# Import necessary libraries
import pandas as pd
import numpy as np
```

```
# Load the Iris dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
column_names = ['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth', 'Species']
df = pd.read_csv(url, header=None, names=column_names)
```

Explanation:

- We load the **Iris dataset** from the provided URL using **pandas.read_csv()** and specify the column names.
-

2. Filter Data for Different Species

We will filter the dataset for specific species ('**Iris-setosa**', '**Iris-versicolor**', '**Iris-virginica**') to calculate statistics separately for each species.

```
# Filter data for different species
setosa = df[df['Species'] == 'Iris-setosa']
versicolor = df[df['Species'] == 'Iris-versicolor']
virginica = df[df['Species'] == 'Iris-virginica']
```

Explanation:

- We create separate DataFrames for each species using boolean indexing. This allows us to perform calculations on each species individually.
-

3. Calculate Basic Statistical Measures

For each species, we will calculate the following descriptive statistics:

- **Mean:** The average of each feature.
- **Standard Deviation:** The variability of each feature.
- **Percentiles (25th, 50th, 75th):** The distribution of values.
- **Min and Max:** The minimum and maximum values.
- **Count:** The number of observations.

```
# Statistical details for Iris-setosa
setosa_stats = setosa.describe()
print("Iris-setosa Statistics:\n", setosa_stats)

# Statistical details for Iris-versicolor
versicolor_stats = versicolor.describe()
print("\nIris-versicolor Statistics:\n", versicolor_stats)

# Statistical details for Iris-virginica
virginica_stats = virginica.describe()

print("\nIris-virginica Statistics:\n", virginica_stats)
```

Explanation:

- The `describe()` function in `pandas` provides various statistics, including the count, mean, standard deviation, min, 25th percentile (Q1), median (50th percentile), 75th percentile (Q3), and max for each numeric column.
- We calculate these statistics separately for each species.

4. Percentile Calculation

In addition to the `describe()` function, we can manually calculate specific percentiles for each species if needed.

```
# Calculate specific percentiles (e.g., 10th, 90th percentile)
percentile_10_setosa = np.percentile(setosa['SepalLength'], 10)
percentile_90_setosa = np.percentile(setosa['SepalLength'], 90)
print(f"\n10th Percentile of Sepal Length (Iris-setosa): {percentile_10_setosa}")
print(f"90th Percentile of Sepal Length (Iris-setosa): {percentile_90_setosa}")
```

Explanation:

- `np.percentile()` is used to calculate any specific percentile (e.g., 10th, 90th) for the given feature (`SepalLength`) of the species.

5. Visualizing the Data (Optional)

To better understand the data and its distribution, we can visualize the statistics using histograms or boxplots.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Plot histogram for SepalLength of Iris-setosa
sns.histplot(setosa['SepalLength'], kde=True)
plt.title('Distribution of Sepal Length (Iris-setosa)')
plt.show()
```

Explanation:

- The `histplot()` function is used to visualize the distribution of the `SepalLength` feature of `Iris-setosa`. We use a Kernel Density Estimate (KDE) for smoother visualization.

Important Coding Concepts

1. **Descriptive Statistics:** Basic statistical measures used to summarize data (mean, median, standard deviation, percentiles).
 2. **Filtering Data:** Using boolean indexing to select specific subsets of data (e.g., by species).
 3. **Percentiles:** A statistical measure indicating the relative standing of a value in a dataset.
 4. **Data Visualization:** Using plots like histograms and boxplots to better understand data distribution.
-

Conclusion

In this exercise, we calculated various **descriptive statistics** for the Iris dataset, including measures of central tendency (mean, median) and variability (standard deviation, percentiles). By performing these calculations on the three species (`Iris-setosa`, `Iris-versicolor`, `Iris-virginica`), we gained insights into their distributions and characteristics. Additionally, we visualized the data to better understand the distribution and variability of specific features.

Descriptive statistics are an essential step in data analysis, providing a summary of key metrics that can inform further analysis or model building.

Data Analytics-I: Linear Regression Model for Predicting Home Prices using the Boston Housing Dataset

Objective

The objective of this project is to create a **Linear Regression model** using Python or R to predict the price of houses based on the features provided in the **Boston Housing Dataset**. The dataset contains information about various houses in Boston, such as the number of rooms, age of the property, distance to employment centers, and more. The goal is to predict the target variable, **house prices**, based on these features.

Hardware Requirements

- **Processor:** Any modern processor (Intel i5 or higher recommended)
 - **RAM:** 4GB or more (8GB or higher recommended)
 - **Storage:** 10GB or more free space (for storing datasets and Python/R environment)
-

Software Requirements

- **Python (version 3.x) or R (version 4.x)**
 - **Libraries/Modules:**
 - **Python:**
 - **pandas** for data manipulation
 - **numpy** for numerical operations
 - **matplotlib** and **seaborn** for data visualization
 - **scikit-learn** for linear regression modeling
 - **statsmodels** (optional for detailed statistical modeling)
-

Theory

Linear Regression

Linear regression is a statistical method that models the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a linear equation to

observed data. The goal is to find the line of best fit, which minimizes the residual sum of squares between the observed values and the values predicted by the model.

In the case of the **Boston Housing Dataset**, the dependent variable is the **price of the house**, and the independent variables (features) include characteristics such as:

- **CRIM**: Crime rate by town
- **ZN**: Proportion of residential land zoned for large plots
- **INDUS**: Proportion of non-retail business acres per town
- **CHAS**: Charles River dummy variable (1 if tract bounds river, 0 otherwise)
- **NOX**: Nitrogen oxide concentration (parts per 10 million)
- **RM**: Average number of rooms per dwelling
- **AGE**: Proportion of homes built before 1940
- **DIS**: Weighted distance to employment centers
- **RAD**: Index of accessibility to radial highways
- **TAX**: Property tax rate per \$10,000
- **PTRATIO**: Pupil-teacher ratio by town
- **B**: Proportion of residents of African American descent
- **LSTAT**: Percentage of lower status population
- **MEDV**: Median value of owner-occupied homes (target variable)

Linear regression assumes a linear relationship between the independent and dependent variables. By fitting a regression model, we can estimate the value of the dependent variable based on the independent features.

Steps

1. Load and Explore the Dataset

We will first load the **Boston Housing Dataset** and explore it. You can download the dataset from Kaggle [Boston Housing Dataset](#).

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
# Load the dataset (URL from Kaggle or local CSV file)
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.csv'
columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B',
'LSTAT', 'MEDV']
df = pd.read_csv(url, names=columns)

# Check for missing values and inspect the first few rows
print(df.isnull().sum())
print(df.head())
```

Explanation:

- We load the dataset using `pandas.read_csv()`.
 - The dataset is checked for missing values and the first few rows are displayed for inspection.
-

2. Data Visualization and Exploration

Visualizing the dataset will help in understanding the relationships between features and the target variable (`MEDV`).

```
# Visualize the correlation between features and the target (MEDV)
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title("Correlation Matrix")
plt.show()
```

Explanation:

- We use a heatmap to visualize the correlation between the features and the target variable (`MEDV`). This helps us understand which features are more strongly related to house prices.
-

3. Prepare the Data for Training

We split the dataset into **features (X)** and the **target (y)**. Then, we divide the data into training and testing sets.

```
# Features and target variable
X = df.drop('MEDV', axis=1) # Features (independent variables)
y = df['MEDV'] # Target (dependent variable)
```

```
# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Explanation:

- We drop the target variable (**MEDV**) from the features dataset (**X**) and assign it to **y**.
- We then split the dataset into training (80%) and testing (20%) sets using **train_test_split()** from **scikit-learn**.

4. Train the Linear Regression Model

Now, we will train the **Linear Regression** model on the training data.

```
# Initialize and train the linear regression model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
# Predict the target values on the test set
y_pred = model.predict(X_test)
```

Explanation:

- We initialize a **LinearRegression** model and fit it to the training data (**X_train** and **y_train**).
 - After training, we predict house prices (**y_pred**) on the testing data (**X_test**).
-

5. Evaluate the Model

We evaluate the model using **Mean Squared Error (MSE)** and **R-squared (R²)** metrics to assess the performance of the model.

```
# Calculate Mean Squared Error (MSE) and R-squared (R2)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}")
```

Explanation:

- **Mean Squared Error (MSE)** quantifies the average of the squared differences between the predicted values and the actual values. Lower values indicate better performance.

- **R-squared (R^2)** represents the proportion of the variance in the dependent variable that is predictable from the independent variables. An R^2 value closer to 1 indicates that the model fits the data well.
-

6. Model Evaluation and Results Visualization

Finally, we can visualize the predicted house prices against the actual prices to check how well the model performed.

```
# Visualize the predicted vs actual prices
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linewidth=2)
plt.title("Predicted vs Actual Prices")
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.show()
```

Explanation:

- We plot the actual house prices (**y_test**) against the predicted house prices (**y_pred**). A perfect prediction would lie along the red line, indicating a strong correlation between the actual and predicted prices.
-

Important Coding Concepts

1. **Linear Regression:** A statistical method for modeling the relationship between a dependent variable and one or more independent variables.
 2. **Train-Test Split:** Dividing the dataset into training and testing sets to evaluate model performance on unseen data.
 3. **Mean Squared Error (MSE):** A metric used to evaluate the performance of a regression model.
 4. **R-squared (R^2):** A metric that indicates how well the regression model explains the variance in the target variable.
 5. **Data Preprocessing:** The steps taken to prepare the data for model training, including cleaning, feature selection, and splitting the dataset.
-

Conclusion

In this exercise, we developed a **Linear Regression** model to predict house prices using the **Boston Housing Dataset**. By visualizing the data and analyzing the relationships between features and the target variable, we were able to create a model that predicts house prices based on various attributes of the properties. The evaluation metrics, such as **Mean Squared Error (MSE)** and **R-squared (R^2)**, helped assess the model's performance, and visualization allowed us to visually confirm the accuracy of our predictions.

This exercise demonstrates how linear regression can be used as a powerful tool in predictive analytics, offering valuable insights into real estate and housing markets.

Data Analytics-II

Project Title: Implementing Logistic Regression and Computing Performance Metrics

Objective:

The primary objective of this project is to implement logistic regression using Python or R to perform classification on the *Social_Network_Ads.csv* dataset. Additionally, we will compute key performance metrics using a confusion matrix, including True Positive (TP), False Positive (FP), True Negative (TN), False Negative (FN), Accuracy, Error Rate, Precision, and Recall.

Hardware Requirements:

- **Processor:** Intel i3 or higher (recommended i5/i7)
- **RAM:** 4GB minimum (8GB recommended)
- **Hard Disk:** 50 GB of free space for data storage and model execution
- **Graphics:** Integrated graphics sufficient for data processing tasks
- **Network:** A stable internet connection for downloading libraries and dataset

Software Requirements:

- **Python** or **R** (preferably Python 3.x)
- **Python Libraries (for Python):**
 - **pandas** for data manipulation
 - **numpy** for numerical operations
 - **matplotlib/seaborn** for data visualization
 - **sklearn** for logistic regression and evaluation metrics
- **Jupyter Notebook** (Python) or **RStudio** (R)
- **Operating System:** Windows/Linux/Mac OS

Theory:

Logistic Regression is a statistical method for binary classification problems. It is used to model the probability of a binary response (i.e., outcome variable with two possible values like 0 and 1). Logistic regression uses the sigmoid function to map predicted values between 0 and 1, which can be interpreted as probabilities.

Key Concepts:

1. **Sigmoid Function:** Converts any real-valued number into a value between 0 and 1.
$$\sigma(z) = 1 / (1 + e^{(-z)})$$

2. **Logistic Regression Model:** It aims to find the best-fitting curve (or decision boundary) that separates data points belonging to one class from those belonging to another.
3. **Confusion Matrix:** A performance measurement tool for classification problems, especially binary classification. It consists of:
 - **True Positives (TP):** Correctly predicted positive class
 - **True Negatives (TN):** Correctly predicted negative class
 - **False Positives (FP):** Incorrectly predicted positive class (Type I Error)
 - **False Negatives (FN):** Incorrectly predicted negative class (Type II Error)

Key Performance Metrics:

- **Accuracy:** The proportion of correctly predicted instances:
 $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$
- **Error Rate:** The proportion of incorrectly predicted instances: Error
 $\text{Rate} = (\text{FP} + \text{FN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$
- **Precision:** The proportion of positive predictions that are correct: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- **Recall (Sensitivity):** The proportion of actual positive instances correctly predicted:
 $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

Important Coding Concepts:

1. **Data Preprocessing:**
 - Handling missing values
 - Encoding categorical variables
 - Splitting the dataset into training and testing sets (e.g., 70% for training and 30% for testing)
2. **Model Building:**
 - Fitting the logistic regression model using training data
 - Applying the logistic function to predict probabilities
3. **Model Evaluation:**
 - Using `confusion_matrix()` to compute TP, TN, FP, FN
 - Using `accuracy_score()`, `precision_score()`, and `recall_score()` to compute other metrics
4. **Visualization:**
 - Plotting confusion matrix as a heatmap
 - Visualizing the decision boundary of the logistic regression model (for 2D data)

Code (Python Example):

```
# Importing Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

# Loading Dataset
dataset = pd.read_csv('Social_Network_Ads.csv')

# Preprocessing
X = dataset.iloc[:, 2:-1].values # Features
y = dataset.iloc[:, -1].values # Target variable (Purchased)

# Splitting the dataset into Training and Test Set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Logistic Regression Model
classifier = LogisticRegression(random_state=42)
classifier.fit(X_train, y_train)

# Predicting the Test set results
y_pred = classifier.predict(X_test)

# Confusion Matrix and Performance Metrics
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
error_rate = 1 - accuracy

print("Confusion Matrix:\n", cm)
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"Error Rate: {error_rate}")
```

Conclusion:

In this project, we implemented logistic regression to classify data from the *Social_Network_Ads.csv* dataset. We successfully computed the key performance metrics like accuracy, precision, recall, and error rate using a confusion matrix. Logistic regression proved to be an efficient model for this binary classification task. The results of the confusion matrix and metrics help us evaluate the model's performance and make necessary improvements if needed. By visualizing these metrics, we can better understand how well the model is predicting the classes.

Data Analytics-III

Project Title: Implementing Naïve Bayes Classification and Computing Performance Metrics

Objective:

The objective of this project is to implement the **Naïve Bayes** classification algorithm using Python or R to classify the data from the *iris.csv* dataset. In addition, we will compute performance metrics such as the **Confusion Matrix**, **True Positive (TP)**, **False Positive (FP)**, **True Negative (TN)**, **False Negative (FN)**, **Accuracy**, **Error Rate**, **Precision**, and **Recall**.

Hardware Requirements:

- **Processor:** Intel i3 or higher (i5/i7 recommended for better performance)
- **RAM:** 4GB minimum (8GB recommended for larger datasets)
- **Hard Disk:** 50 GB of free space
- **Graphics:** Integrated graphics suffice for data processing tasks
- **Network:** A stable internet connection for downloading the required libraries and dataset

Software Requirements:

- **Python** or **R** (preferably Python 3.x)
- **Python Libraries (for Python):**
 - **pandas** for data manipulation
 - **numpy** for numerical operations
 - **scikit-learn** for Naïve Bayes and evaluation metrics
 - **matplotlib/seaborn** for visualization
- **Jupyter Notebook** (Python) or **RStudio** (R)
- **Operating System:** Windows/Linux/Mac OS

Theory:

Naïve Bayes is a probabilistic classifier based on applying **Bayes' Theorem** with the **Naïve** assumption that the features are independent given the class label. Despite its simplicity, it is highly effective, especially for classification tasks where the independence assumption approximately holds.

Bayes' Theorem:

Bayes' Theorem allows us to compute the probability of a class (label) given the observed features (predictors):

$$P(C|X)=P(X|C)P(C) / P(X)$$

Where:

- $P(C|X)$ is the posterior probability of the class given the predictors
- $P(X|C)$ is the likelihood of observing the predictors given the class
- $P(C)$ is the prior probability of the class
- $P(X)$ is the marginal probability of the predictors

Naïve Assumption:

Naïve Bayes assumes that each feature is independent given the class label, simplifying the calculation of $P(X|C)$ as the product of individual probabilities of each feature:

$$P(X|C) = P(x_1|C) \times P(x_2|C) \times \dots \times P(x_n|C)$$

This assumption, though simplistic, often leads to surprisingly effective results, especially in text classification and smaller datasets.

Confusion Matrix:

A **confusion matrix** is a table used to evaluate the performance of a classification algorithm. It provides insights into:

- **True Positives (TP):** Correctly predicted positive class instances
- **True Negatives (TN):** Correctly predicted negative class instances
- **False Positives (FP):** Instances incorrectly predicted as positive
- **False Negatives (FN):** Instances incorrectly predicted as negative

Performance Metrics:

- **Accuracy:** The proportion of correctly predicted instances:
 $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$
- **Error Rate:** The proportion of incorrectly predicted instances: Error
 $\text{Rate} = (FP + FN) / (TP + TN + FP + FN)$
- **Precision:** The proportion of positive predictions that are correct: $\text{Precision} = TP / (TP + FP)$
- **Recall (Sensitivity):** The proportion of actual positive instances correctly predicted:
 $\text{Recall} = TP / (TP + FN)$

Important Coding Concepts:

1. Data Preprocessing:

- Loading and cleaning the dataset (handling missing values, if any)
- Splitting the dataset into features and labels
- Splitting the data into training and testing sets
- Feature scaling is not required for Naïve Bayes as it is not sensitive to the scale of the data

2. Model Building:

- Using Naïve Bayes for classification using **GaussianNB** or **MultinomialNB** depending on the nature of the features

3. Model Evaluation:

- Use the **confusion matrix** to evaluate the number of correct and incorrect predictions
- Compute **accuracy**, **precision**, **recall**, and **error rate**

4. Visualization:

- Visualize the confusion matrix as a heatmap
- Plotting the decision boundaries or feature distributions for insight

Code (Python Example):

```
# Importing Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
import seaborn as sns
import matplotlib.pyplot as plt

# Loading Dataset
dataset = pd.read_csv('iris.csv')

# Preprocessing: Splitting features and target
X = dataset.iloc[:, :-1].values # Features (all columns except last)
y = dataset.iloc[:, -1].values # Target (species)

# Splitting the dataset into Training and Test Set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Naïve Bayes Model (GaussianNB for continuous features)
model = GaussianNB()
model.fit(X_train, y_train)

# Making Predictions
y_pred = model.predict(X_test)

# Confusion Matrix and Performance Metrics
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
error_rate = 1 - accuracy

print("Confusion Matrix:\n", cm)
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"Error Rate: {error_rate}")
```

```
# Visualizing the Confusion Matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=model.classes_,
yticklabels=model.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

Conclusion:

In this project, we implemented the **Naïve Bayes** classification algorithm to predict the species of flowers in the *iris.csv* dataset. We evaluated the model's performance using various metrics such as **accuracy**, **precision**, **recall**, and **error rate**, and visualized the **confusion matrix** to assess the model's effectiveness. Naïve Bayes performed well with the iris dataset due to its simplicity and efficiency, especially given that the features (sepal length, width, petal length, and width) exhibit conditional independence, which aligns well with the assumptions of the algorithm.

Text Analytics

Project Title: Document Preprocessing and Term Frequency-Inverse Document Frequency (TF-IDF) Calculation

Objective:

The primary goal of this project is to extract a sample document, apply various text preprocessing methods, including **Tokenization**, **Part of Speech (POS) Tagging**, **Stop Words Removal**, **Stemming**, and **Lemmatization**. Additionally, we will calculate **Term Frequency (TF)** and **Inverse Document Frequency (IDF)** to create a representation of the documents.

Hardware Requirements:

- **Processor:** Intel i3 or higher (i5/i7 recommended for better performance)
- **RAM:** 4GB minimum (8GB recommended for larger datasets)
- **Hard Disk:** 50 GB of free space
- **Graphics:** Integrated graphics suffice for text processing tasks
- **Network:** Stable internet connection for downloading datasets and required libraries

Software Requirements:

- **Python** or **R** (preferably Python 3.x)
- **Python Libraries (for Python):**
 - **nltk** for text preprocessing (tokenization, POS tagging, stop words removal, stemming, and lemmatization)
 - **sklearn** for calculating TF and IDF
 - **pandas** for handling text data
 - **matplotlib** or **seaborn** for data visualization (optional)
- **Jupyter Notebook** (Python) or **RStudio** (R)
- **Operating System:** Windows/Linux/Mac OS

Theory:

Text analytics is a field of Natural Language Processing (NLP) that focuses on extracting meaningful information from unstructured text. The goal of text preprocessing is to clean and structure the text to make it ready for further analysis or modeling. Below are the key concepts:

Document Preprocessing:

1. **Tokenization:** Tokenization is the process of splitting text into smaller units (tokens), which could be words or sub-words. It is a crucial first step in NLP tasks, as it helps in breaking down text into manageable pieces.

Example:

- Input: "Text analytics is fun!"
 - Output: ["Text", "analytics", "is", "fun"]
2. **POS (Part of Speech) Tagging:** POS tagging is the process of identifying the grammatical category (such as noun, verb, adjective) of each token. This helps in understanding the syntactic structure of sentences.

Example:

- Input: "Text analytics is fun!"
 - Output: [('Text', 'NN'), ('analytics', 'NNS'), ('is', 'VBZ'), ('fun', 'NN')]
3. **Stop Words Removal:** Stop words are common words (e.g., 'the', 'is', 'in') that do not contribute much meaning to the analysis and are often removed during preprocessing to reduce noise.

Example:

- Input: "This is a simple example."
 - Output: ["simple", "example"]
4. **Stemming:** Stemming is the process of reducing words to their root form, which may not always be a valid word. For example, "running" becomes "run".

Example:

- Input: "Running, runner"
 - Output: ["run", "run"]
5. **Lemmatization:** Lemmatization is similar to stemming, but it ensures that the root word is a valid word by using a vocabulary. For example, "running" becomes "run" while also ensuring the word is a valid lemma in the dictionary.

Example:

- Input: "Running, better"
- Output: ["run", "good"]

Term Frequency and Inverse Document Frequency (TF-IDF):

TF-IDF is a statistical measure used to evaluate the importance of a word in a document relative to a collection of documents (corpus). It balances two components:

1. **Term Frequency (TF):** Measures how frequently a word appears in a document.
$$TF(t) = \frac{\text{Count of term } t \text{ in document}}{\text{Total terms in the document}}$$

$$TF(t) = (\text{Count of term } t \text{ in document}) / (\text{Total terms in the document})$$

2. **Inverse Document Frequency (IDF):** Measures how important a word is across the entire corpus. Words that appear frequently in many documents have a low IDF, while words that appear in fewer documents have a high IDF.

$idf(t) = \log_{10}(\text{Total number of documents} / \text{Number of documents containing term } t)$

3. **TF-IDF** is the product of TF and IDF and is used to calculate the weight of each word in a document.

$TF-IDF(t) = TF(t) \times IDF(t)$

Important Coding Concepts:

1. Text Preprocessing Steps:

- **Tokenization:** Splitting text into words using `nltk.word_tokenize()` or `nltk.wordpunct_tokenize()`.
- **POS Tagging:** Using `nltk.pos_tag()` to tag each token with its grammatical category.
- **Stop Words Removal:** Using `nltk.corpus.stopwords` to remove common stop words.
- **Stemming and Lemmatization:** Using `nltk.stem.PorterStemmer()` for stemming and `nltk.WordNetLemmatizer()` for lemmatization.

2. TF-IDF Calculation:

- Use the `TfidfVectorizer` from `sklearn.feature_extraction.text` to calculate the TF-IDF for a collection of documents.
- Visualize the TF-IDF matrix using word clouds or bar charts.

Code (Python Example):

```
# Importing Libraries
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer

# Downloading necessary NLTK data files
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

# Sample Document
document = "Text Analytics is a crucial step in analyzing unstructured data. It involves tokenization, lemmatization, and stop words removal."

# Tokenization
tokens = word_tokenize(document)
print("Tokens:", tokens)
```

```
# POS Tagging
pos_tags = nltk.pos_tag(tokens)
print("POS Tags:", pos_tags)

# Stop Words Removal
stop_words = set(stopwords.words('english'))

filtered_tokens = [word for word in tokens if word.lower() not in stop_words]
print("Filtered Tokens:", filtered_tokens)

# Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in filtered_tokens]
print("Stemmed Words:", stemmed_words)

# Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_tokens]
print("Lemmatized Words:", lemmatized_words)

# TF-IDF Calculation
corpus = [document] # List of documents
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(corpus)

# Displaying TF-IDF scores
print("TF-IDF Matrix:")
print(tfidf_matrix.toarray())
print("Feature Names (words):", vectorizer.get_feature_names_out())
```

Conclusion:

In this project, we successfully implemented several document preprocessing techniques, including **Tokenization**, **POS Tagging**, **Stop Words Removal**, **Stemming**, and **Lemmatization** using Python or R. These techniques are essential in preparing textual data for further analysis or machine learning tasks. We also computed **Term Frequency (TF)** and **Inverse Document Frequency (IDF)** to create a representation of the document using the **TF-IDF** metric. This representation helps in evaluating the importance of words in a document relative to a collection of documents. Text preprocessing and vectorization are fundamental steps in any text analytics project, enabling more accurate and efficient analysis of unstructured text data.

Data Visualization I

Project Title: Titanic Dataset Exploration and Fare Distribution Visualization

Objective:

The primary objective of this project is to explore the inbuilt **Titanic dataset** and use the **Seaborn library** for data visualization to find any patterns in the data. Additionally, we will plot a **histogram** to visualize how the ticket prices (column name: 'fare') for each passenger are distributed.

Hardware Requirements:

- **Processor:** Intel i3 or higher (i5/i7 recommended for better performance)
- **RAM:** 4GB minimum (8GB recommended for handling larger datasets)
- **Hard Disk:** 50 GB of free space
- **Graphics:** Integrated graphics suffice for visualization tasks
- **Network:** Stable internet connection for downloading libraries and datasets

Software Requirements:

- **Python 3.x**
- **Python Libraries:**
 - **seaborn** for data visualization
 - **matplotlib** for additional plotting functionalities
 - **pandas** for data manipulation
 - **numpy** for numerical operations
 - **scipy** (optional, for advanced statistical analysis)
- **Jupyter Notebook** or **VS Code** for running Python code
- **Operating System:** Windows/Linux/Mac OS

Theory:

Data visualization is an essential technique for analyzing and interpreting large datasets. It allows us to identify patterns, trends, and outliers, and is especially helpful when dealing with unstructured or complex data. In this project, we will work with the **Titanic dataset**, a commonly used dataset that contains information about the passengers of the Titanic ship. The dataset includes various attributes such as **age**, **sex**, **fare**, **class**, **embarked**, and **survival**.

We will use the **Seaborn library** to explore relationships between the variables and visualize the **fare distribution** using a **histogram**.

Titanic Dataset Overview:

The Titanic dataset contains the following columns:

- **PassengerId**: The passenger's unique ID.
- **Survived**: Whether the passenger survived (1 = Yes, 0 = No).
- **Pclass**: The passenger's class (1st, 2nd, or 3rd).
- **Name**: The passenger's name.
- **Sex**: The gender of the passenger.
- **Age**: The age of the passenger.
- **SibSp**: The number of siblings/spouses aboard the Titanic.
- **Parch**: The number of parents/children aboard the Titanic.
- **Fare**: The ticket price paid by the passenger.
- **Embarked**: The port at which the passenger boarded (C = Cherbourg, Q = Queenstown, S = Southampton).

Data Visualization Techniques:

1. **Seaborn**: Seaborn is a Python library based on Matplotlib that simplifies data visualization. It provides functions to generate beautiful, informative graphics with minimal code. In this project, Seaborn will be used to:
 - Visualize the distribution of ticket prices using a histogram.
 - Explore patterns in data through other types of plots (such as bar plots, scatter plots, etc.).
2. **Histogram**: A histogram is a graphical representation of the distribution of a dataset. It groups data into bins and shows the frequency of data points within each bin. Histograms are particularly useful for understanding the distribution of continuous data like ticket prices.

Important Coding Concepts:

1. **Data Loading and Exploration**:
 - Loading the Titanic dataset using Seaborn's `load_dataset()` function.
 - Checking the first few rows of the dataset using `head()`.
 - Inspecting summary statistics using `describe()` and `info()`.
2. **Data Visualization**:
 - Using Seaborn's `histplot()` to plot the histogram of the 'fare' column.
 - Customizing histograms by setting parameters like `bins` for controlling the number of bins, and `kde` for overlaying a Kernel Density Estimate (smooth curve).
3. **Plot Customization**:
 - Adding labels, titles, and gridlines to improve the readability of the plot.

- Setting figure size using Matplotlib's `plt.figure()`.

4. Exploratory Data Analysis (EDA):

- Using visualizations to check for patterns and insights in the data.
- Identifying the distribution of ticket prices and seeing if there are any obvious patterns (such as outliers or skewed distributions).

Code (Python Example):

```
# Importing Libraries
import seaborn as sns
import matplotlib.pyplot as plt

# Loading the Titanic dataset
titanic_data = sns.load_dataset('titanic')

# Displaying the first few rows of the dataset
print(titanic_data.head())

# Checking summary statistics of the dataset
print(titanic_data.describe())

# Plotting the distribution of ticket prices ('fare') using a histogram
plt.figure(figsize=(10, 6))
sns.histplot(titanic_data['fare'], kde=True, bins=30, color='blue')

# Customizing the plot
plt.title('Distribution of Ticket Prices (Fare) for Titanic Passengers', fontsize=16)
plt.xlabel('Fare', fontsize=14)
plt.ylabel('Frequency', fontsize=14)
plt.grid(True)

# Displaying the plot
plt.show()
```

Explanation of the Code:

1. **Loading Data:** We load the Titanic dataset using `seaborn.load_dataset('titanic')`. This dataset is included in Seaborn's sample datasets, so we don't need to manually download it.
2. **Displaying Data:** We display the first few rows using `head()` and summary statistics using `describe()`. This gives us an understanding of the dataset's structure and data distribution.
3. **Plotting Histogram:** We use `sns.histplot()` to plot a histogram of the 'fare' column. The `kde=True` parameter overlays a kernel density estimate (smooth curve) to give a

clearer view of the distribution. We specify `bins=30` to set the number of bins for the histogram.

4. **Plot Customization:** We add a title and axis labels to the plot for clarity. The grid is also enabled to make the plot easier to interpret.

Code (Expected Output):

- The output will show a histogram representing the distribution of the ticket prices ('fare') of Titanic passengers.
- The histogram will indicate how fares are distributed across different passengers, and the KDE curve will show the smooth density of the fare distribution.

Conclusion:

In this project, we used the **Titanic dataset** to explore the distribution of ticket prices using **Seaborn's `histplot()`** function. The histogram helps us understand how ticket fares were distributed among passengers. By overlaying a Kernel Density Estimate (KDE), we can observe the overall distribution more smoothly, which may reveal skewness or the presence of outliers.

Data visualization plays a vital role in **Exploratory Data Analysis (EDA)**, as it helps us uncover patterns, trends, and outliers that might not be immediately obvious from raw data. This simple visualization can lead to further analyses, such as investigating correlations between fares and other features like **passenger class**, **age**, or **survival rate**.

Data Visualization II: Box Plot for Distribution of Age with Respect to Gender and Survival

Objective

The objective of this task is to visualize the distribution of age in the Titanic dataset based on gender, and also to analyze the age distribution with respect to whether passengers survived or not. A box plot will be used to visually understand the data distribution and infer patterns.

Hardware Requirements

- A computer or laptop capable of running Python and data analysis tools.
- Minimum of 4GB RAM for smooth data processing.

Software Requirements

- Python 3.x or higher
- Libraries:
 - Pandas
 - Matplotlib
 - Seaborn
 - NumPy
- Jupyter Notebook or any other Integrated Development Environment (IDE) like PyCharm or VS Code for coding.

Theory

- **Box Plot:** A box plot is a graphical representation of data spread and highlights the central tendency, variability, and outliers within a dataset. It includes the median, quartiles, and potential outliers in the data. This makes it useful for comparative analysis, such as age distribution across gender or survival status.
- **Titanic Dataset:** The dataset contains information on passengers aboard the Titanic, with columns such as:
 - **Age:** The age of passengers.
 - **Sex:** The gender of the passengers (male or female).
 - **Survived:** Whether the passenger survived (1) or not (0).
- **Purpose of this Visualization:** This analysis aims to compare the age distribution across gender (male and female) and survival status (survived vs. not survived) to infer

insights about the impact of these factors on the survival rate.

Important Coding Concepts

1. Pandas:

- Data manipulation, such as accessing and filtering specific columns (**age**, **sex**, **survived**).
- Data preprocessing, such as handling missing values (if necessary).

2. Matplotlib & Seaborn:

- Creating box plots to visualize the distribution of continuous data (age).
- Customizing the box plot to group data by gender and survival status.

3. Grouping Data:

- Use **groupby** in Pandas to organize the data by gender and survival status for comparison.

4. Plot Customization:

- Customize axes, titles, and labels to make the plot clear and informative.

Code Implementation

```
# Importing necessary libraries
```

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Load the Titanic dataset
```

```
titanic_data = sns.load_dataset('titanic')
```

```
# Plotting box plot for age distribution by gender and survival status
```

```
plt.figure(figsize=(10, 6))
```

```
sns.boxplot(data=titanic_data, x='sex', y='age', hue='survived')
```

```
# Adding labels and title
```

```
plt.title('Box Plot of Age Distribution by Gender and Survival Status')
```

```
plt.xlabel('Gender')
```

```
plt.ylabel('Age')
```

```
plt.legend(title='Survived', loc='upper right', labels=['Did Not Survive', 'Survived'])
```

```
# Display the plot
```

```
plt.show()
```

Explanation of the Code:

1. **Loading Data:** The Titanic dataset is loaded using Seaborn's `load_dataset` function.
2. **Box Plot Creation:** The `sns.boxplot()` function creates the box plot, with:
 - `x='sex'`: Gender (Male, Female).
 - `y='age'`: Age of passengers.
 - `hue='survived'`: Survival status, which differentiates colors in the box plot.
3. **Plot Customization:** Labels and titles are added to make the plot informative.

Observations and Inferences

- **Age Distribution Across Gender:**
 - For both males and females, the age distribution appears skewed, with the majority of passengers being younger, but with a noticeable age range.
- **Survival and Gender:**
 - Females tend to have a wider age distribution, while males are more concentrated in a specific age range.
- **Age and Survival Status:**
 - Passengers who survived (`hue=1`) tend to be younger than those who did not survive (`hue=0`).
 - This suggests that age could have been a contributing factor in survival, with children and younger adults having a higher chance of survival.
- **Outliers:**
 - There are a few outliers in the age distribution, particularly for older passengers, both male and female, who did not survive. This could suggest potential outliers or other factors influencing survival.

Conclusion

The analysis of age distribution in the Titanic dataset with respect to gender and survival status reveals some notable patterns:

- Younger passengers were more likely to survive, with a more concentrated distribution in the younger age range.
- Gender also plays a role in survival rates, with females exhibiting a slightly higher survival rate compared to males.
- The box plot is an effective way to compare the distributions and can highlight patterns that may not be immediately apparent from raw data alone.

This visualization approach is helpful in understanding key variables (age, gender, survival) and their relationships in the context of the Titanic disaster.

Data Visualization III: Exploratory Data Analysis on the Iris Dataset

Objective

The objective of this task is to perform an exploratory data analysis (EDA) on the Iris flower dataset to visualize and analyze the distributions of various features. By using histograms and box plots, we aim to gain insights into the data, identify trends, and recognize potential outliers.

Hardware Requirements

- A computer or laptop capable of running Python and data analysis tools.
- Minimum of 4GB RAM for smooth data processing.

Software Requirements

- Python 3.x or higher
- Libraries:
 - Pandas
 - Matplotlib
 - Seaborn
 - NumPy
- Jupyter Notebook or any other Integrated Development Environment (IDE) like PyCharm or VS Code for coding.

Theory

- **Iris Dataset:** The Iris dataset is a widely used dataset in machine learning and statistics, consisting of 150 observations of iris flowers. The dataset contains the following features:
 - **Sepal Length:** The length of the sepal in centimeters.
 - **Sepal Width:** The width of the sepal in centimeters.
 - **Petal Length:** The length of the petal in centimeters.
 - **Petal Width:** The width of the petal in centimeters.
 - **Species:** The species of the iris (Setosa, Versicolor, or Virginica), which is a categorical (nominal) feature.
- **Histogram:** A histogram is a graphical representation of the distribution of numerical data, showing how often each range of values occurs. It is useful for identifying patterns

such as skewness, peaks, and outliers.

- **Box Plot:** A box plot visualizes the distribution of numerical data through their quartiles, highlighting outliers. It shows the median, upper and lower quartiles, and potential outliers in the data.

Important Coding Concepts

1. Pandas:

- Loading the Iris dataset using `pd.read_csv()` or directly from the UCI repository (depending on source).
- Dataframe exploration using `.head()`, `.info()`, and `.describe()`.
- Identifying the data types of features and their basic statistics.

2. Matplotlib & Seaborn:

- Creating histograms with `sns.histplot()` to visualize the distribution of each feature.
- Creating box plots with `sns.boxplot()` to analyze the spread and outliers in the data.

3. Feature Types:

- Understanding the difference between numerical (continuous) and categorical (nominal) features.

4. Identifying Outliers:

- Using visualizations (box plots) to spot potential outliers in each feature.

Code Implementation

Step 1: Load and Explore the Iris Dataset

```
# Importing necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Iris dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
column_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
iris_data = pd.read_csv(url, names=column_names)

# Displaying the first few rows of the dataset
print(iris_data.head())
print(iris_data.info())
```

Step 2: Feature Types

```
# Displaying feature types and basic statistics
print(iris_data.dtypes)
print(iris_data.describe())
```

Step 3: Histogram for Each Feature

```
# Plotting histograms for each feature
iris_data.hist(bins=10, figsize=(10, 8))
plt.suptitle('Histograms of Iris Dataset Features')
plt.show()
```

Step 4: Boxplot for Each Feature

```
# Plotting boxplots for each feature
plt.figure(figsize=(10, 8))
sns.boxplot(data=iris_data.drop('species', axis=1))
plt.title('Box Plot of Features in the Iris Dataset')
plt.show()
```

Step 5: Compare Distributions and Identify Outliers

```
# Visualizing histograms and boxplots for each feature grouped by species
plt.figure(figsize=(12, 8))
for i, feature in enumerate(iris_data.columns[:-1]):
    plt.subplot(2, 2, i + 1)
    sns.histplot(data=iris_data, x=feature, hue="species", kde=True)
    plt.title(f'Distribution of {feature}')

plt.tight_layout()
plt.show()
```

```
# Comparing boxplots for different species
plt.figure(figsize=(12, 8))
for i, feature in enumerate(iris_data.columns[:-1]):
    plt.subplot(2, 2, i + 1)
    sns.boxplot(x='species', y=feature, data=iris_data)
    plt.title(f'Box Plot of {feature} by Species')

plt.tight_layout()
plt.show()
```

Explanation of the Code:

1. **Loading Data:** The dataset is loaded from the UCI repository, and columns are named appropriately.
2. **Feature Types:** We inspect the data types using `dtypes` and summarize basic statistics with `describe()`.
3. **Histograms:** We plot histograms for each feature to observe the distribution of values.
4. **Boxplots:** Box plots are drawn for each feature to visually inspect the spread of data, quartiles, and outliers.
5. **Comparison of Distributions:** We use both histograms and box plots grouped by species to compare distributions across the different iris species.

Observations and Inferences

- **Features and Types:**
 - **Sepal Length, Sepal Width, Petal Length, Petal Width** are all **numerical** features (continuous data).
 - **Species** is a **categorical** feature (nominal data), representing different iris flower species.
- **Histograms:**
 - **Sepal Length** and **Petal Length** have relatively normal distributions, while **Sepal Width** and **Petal Width** are slightly skewed.
 - Petal width and petal length show higher concentration in specific value ranges, while sepal length has a broader range.
- **Boxplots:**
 - **Sepal Length** shows a wider range for Setosa and Virginica species, while Versicolor is more tightly grouped.
 - **Petal Length** and **Petal Width** show clear separation between species, with Setosa having smaller values compared to the other two species.
 - **Outliers:** The box plots for sepal width show potential outliers for certain species, though the dataset as a whole doesn't have extreme outliers.
- **Comparisons by Species:**
 - **Setosa** generally has smaller values for both petal length and width, while **Virginica** has the largest values for these features.
 - **Versicolor** is typically in between the two, suggesting it is a middle ground between Setosa and Virginica in terms of petal measurements.

Conclusion

Through the exploratory data analysis (EDA) of the Iris dataset, we can conclude the following:

- The dataset provides valuable insights into the physical differences between the species of iris flowers.
- Petal dimensions (length and width) are the best differentiators for the species, while sepal dimensions show more overlap.
- Visualizing the data through histograms and box plots has helped in identifying patterns, distributions, and outliers.
- The Iris dataset is well-suited for classification tasks due to the clear separation between species based on flower features.

This analysis serves as a foundation for further tasks, such as machine learning classification models to predict iris species based on these features.

Creating Databases and Tables, Inserting Data, and Running Simple Queries Using Impala

Objective

The objective of this task is to create a database and tables, insert a small amount of data, and run basic SQL queries using Impala, a high-performance, distributed SQL engine for Apache Hadoop. This exercise aims to help users understand how to interact with the Impala database system, perform basic data operations, and familiarize themselves with querying in a distributed environment.

Hardware Requirements

- A computer or server with at least 4 GB of RAM for basic data operations.
- Minimum 1 CPU core for running the Impala engine.
- Local or cloud-based Hadoop environment to run Impala.

Software Requirements

- **Impala:** A distributed SQL query engine for Hadoop, usually deployed as part of the Cloudera ecosystem.
- **Hadoop Cluster (or a local setup):** Impala relies on the Hadoop Distributed File System (HDFS) for data storage.
- **Impala-shell or Hue:** Command-line interface or a graphical tool for interacting with Impala.
- **JDBC/ODBC client (optional):** For connecting and querying from external applications like BI tools or custom programs.
- **Hive (optional):** If integrating with Hive tables for broader data querying.

Theory

- **Impala Overview:** Impala is an SQL query engine designed for low-latency, high-performance querying over big data. It works with data stored in HDFS and provides a fast, interactive query execution environment. Impala uses a familiar SQL interface and supports a range of SQL operations such as `CREATE DATABASE`, `CREATE TABLE`, `INSERT`, and `SELECT`.
- **Database and Tables:** In Impala, a **database** is a container for managing tables, and **tables** are the structures in which data is stored. Tables in Impala are organized into columns and rows, similar to traditional relational databases.

- **SQL Queries:**

- **CREATE DATABASE:** Used to create a new database.
- **CREATE TABLE:** Defines a new table in the database.
- **INSERT INTO:** Used to insert records into a table.
- **SELECT:** Queries data from tables.
- **DROP DATABASE** and **DROP TABLE:** Used to remove databases and tables.

Important Coding Concepts

1. Impala-SQL Syntax:

- Impala uses SQL-like syntax to interact with the data.

Syntax for basic operations:

```
CREATE DATABASE <db_name>;  
CREATE TABLE <table_name> (<columns definition>);  
INSERT INTO <table_name> VALUES (<values>);  
SELECT * FROM <table_name>;
```

2. Data Types in Impala:

- **String Types:** `STRING`, `VARCHAR`
- **Numeric Types:** `INT`, `FLOAT`, `DOUBLE`, `BIGINT`
- **Date/Time Types:** `DATE`, `TIMESTAMP`
- **Complex Types:** `ARRAY`, `STRUCT`, `MAP`

3. Basic SQL Operations:

- **Create:** Creating databases and tables to store structured data.
- **Insert:** Adding data to tables.
- **Select:** Retrieving data to perform analysis or operations.
- **Alter** and **Drop:** Modifying or deleting databases/tables.

Code Implementation

Step 1: Creating a Database

```
-- Creating a new database  
CREATE DATABASE test_db;
```

Step 2: Creating a Table

```
-- Creating a table within the created database  
USE test_db; -- Switch to the created database
```

```
CREATE TABLE employees (  
    id INT,  
    name STRING,  
    age INT,  
    department STRING  
);
```

Step 3: Inserting Data into the Table

```
-- Inserting some sample data into the table  
INSERT INTO employees VALUES (1, 'Alice', 30, 'HR');  
INSERT INTO employees VALUES (2, 'Bob', 35, 'Engineering');  
INSERT INTO employees VALUES (3, 'Charlie', 40, 'Marketing');
```

Step 4: Querying Data

```
-- Running a simple query to fetch all records from the table  
SELECT * FROM employees;  
  
-- Query to fetch employees from a specific department  
SELECT * FROM employees WHERE department = 'Engineering';
```

Step 5: Dropping the Table and Database (Optional Cleanup)

```
-- Dropping the table and database (useful for cleanup)  
DROP TABLE employees;  
DROP DATABASE test_db;
```

Explanation of the Code:

1. **CREATE DATABASE:** The command `CREATE DATABASE test_db` is used to create a new database named `test_db`.
2. **USE database:** The `USE` command switches to the `test_db` database to ensure that all subsequent operations are performed on this database.
3. **CREATE TABLE:** The table `employees` is created with four columns (`id`, `name`, `age`, `department`) representing employee data.
4. **INSERT INTO:** Records are inserted into the `employees` table using the `INSERT INTO` statement.
5. **SELECT:** Simple queries are performed using the `SELECT` statement to retrieve data from the table. The second query filters the data to display only employees from the "Engineering" department.
6. **DROP TABLE and DROP DATABASE:** These commands clean up the environment by deleting the created table and database.

Observations and Inferences

- **Database and Table Creation:** Creating databases and tables in Impala is straightforward, with a familiar SQL syntax.
- **Data Insertion:** The `INSERT INTO` statement can add data to tables, and in this case, small amounts of sample data were inserted into the `employees` table.
- **Query Execution:** Queries are easy to execute in Impala using SQL syntax, allowing for efficient data retrieval. The query examples show how to fetch all data and filter it based on specific conditions.
- **Performance:** Impala is known for its high performance and low-latency queries, making it suitable for real-time data analysis over large datasets.
- **Simple Data Operations:** These basic operations (CREATE, INSERT, SELECT) form the foundation of working with Impala and similar distributed SQL query engines.

Conclusion

In this task, we successfully created a database and table, inserted small amounts of data, and ran simple queries using Impala. These fundamental operations—database creation, table creation, data insertion, and querying—are essential skills for working with Impala in a distributed environment. Impala's SQL-like syntax makes it easy to interact with data stored in Hadoop, and the system is capable of handling both small and large datasets efficiently. This foundational knowledge sets the stage for more complex data operations and analysis within Impala.

Write a Simple Program in Scala Using Apache Spark Framework

Objective

The objective of this task is to write a simple program in Scala using the Apache Spark framework. The program will load a dataset, perform basic data transformations, and run simple operations like filtering and aggregations to demonstrate Spark's capabilities for big data processing.

Hardware Requirements

- A computer or server with at least 4 GB of RAM for small-scale data processing.
- Minimum of 2 CPU cores for running the Apache Spark engine.
- A Hadoop or Spark cluster or a local installation to run Spark jobs.
- A network connection for downloading dependencies and datasets (if not using local files).

Software Requirements

- **Scala:** A high-level programming language that runs on the Java Virtual Machine (JVM), used for developing Spark applications.
- **Apache Spark:** A distributed data processing framework for big data processing.
- **SBT (Scala Build Tool):** For compiling and managing Scala projects.
- **IDE:** Integrated Development Environment (IDE) like IntelliJ IDEA with the Scala plugin for writing and running Scala code.
- **Hadoop (optional):** To run Spark in a cluster environment (if required).
- **Java:** Java 8 or higher for running Spark jobs (since Spark is built on Java).

Theory

- **Apache Spark Overview:** Apache Spark is an open-source, distributed computing system that provides an interface for programming entire clusters with implicit data parallelism and fault tolerance. It is designed for speed and ease of use, offering APIs in Scala, Python, Java, and R. Spark is widely used for large-scale data processing, analytics, and machine learning.
- **RDDs (Resilient Distributed Datasets):** The fundamental abstraction in Spark. RDDs are distributed collections of objects, which can be processed in parallel across a cluster. They provide fault tolerance, as they can recover lost data using lineage information.
- **Spark SQL:** Provides an interface for working with structured and semi-structured data. It allows querying data using SQL-like syntax and can seamlessly integrate with

Data Frames.

- **Spark Context:** The entry point to Spark functionality, responsible for establishing a connection to the cluster and managing the lifecycle of Spark jobs.
- **Transformation vs. Action:**
 - **Transformations** (e.g., `map`, `filter`, `flatMap`) are operations that return a new RDD and are lazily evaluated.
 - **Actions** (e.g., `collect`, `count`, `reduce`) trigger the execution of transformations and return results.

Important Coding Concepts

1. **SparkContext:** This is the entry point for Spark functionality. It connects to the cluster and manages the execution of jobs.
2. **RDD (Resilient Distributed Dataset):** The core data structure in Spark, which allows parallel processing across multiple nodes in the cluster.
3. **Actions and Transformations:**
 - **Transformations** are operations that build a new dataset (like `map`, `filter`, etc.) from an existing one.
 - **Actions** trigger the execution of the transformations (like `collect`, `count`, etc.) and return results.
4. **DataFrames:** A higher-level abstraction in Spark for working with structured data. DataFrames are similar to tables in a relational database and are built on top of RDDs.
5. **SparkSession:** A unified entry point for reading data, creating DataFrames, and executing SQL queries in Spark.

Code Implementation

Below is a simple Scala program using Apache Spark that reads a dataset, applies transformations, and runs an aggregation.

Step 1: Set up your Spark Application

```
import org.apache.spark.sql.SparkSession
```

```
object SparkExample {  
  def main(args: Array[String]): Unit = {
```

```
// Step 1: Initialize Spark Session
val spark = SparkSession.builder()
    .appName("Simple Spark Program")

    .config("spark.master", "local")
    .getOrCreate()

// Step 2: Load data from a CSV file into a DataFrame
val df = spark.read.option("header", "true").csv("data/people.csv")

// Step 3: Show the first few rows of the DataFrame
df.show()

// Step 4: Perform a simple transformation and action
val ageData = df.filter(df("age") > 30).select("name", "age")

// Step 5: Show the filtered results
ageData.show()

// Step 6: Group by and Aggregate Data
val avgAge = df.groupBy("department").avg("age")
avgAge.show()

// Stop the Spark session
spark.stop()
}
}
```

Explanation of the Code:

1. **SparkSession:** The `SparkSession.builder()` is used to initialize the Spark session, which is the entry point for all Spark functionality. It connects to the Spark cluster and handles both batch and streaming data.
2. **Loading Data:** The `spark.read.option("header", "true").csv("data/people.csv")` command loads a CSV file (e.g., `people.csv`) into a DataFrame, with the first row treated as headers.
3. **Transformations:** The `filter` operation is used to select only the rows where the `age` column is greater than 30, and `select` picks specific columns (`name` and `age`).

4. **Actions:** The `show()` method is called on the DataFrame to display the results. This is an action that triggers the execution of the transformations.
5. **Grouping and Aggregating:** The `groupBy("department").avg("age")` groups the data by the `department` column and calculates the average age in each department.
6. **Stop Spark:** The `spark.stop()` stops the Spark session once the job is completed.

Example Input (`people.csv`):

```
name,age,department
Alice,30,HR
Bob,35,Engineering
Charlie,40,Engineering
David,50,HR
Eve,28,Marketing
```

Expected Output:

```
+-----+---+-----+
| name|age|department|
+-----+---+-----+
| Alice| 30|      HR|
|  Bob| 35|Engineering|
|Charlie| 40|Engineering|
| David| 50|      HR|
|  Eve| 28| Marketing|
+-----+---+-----+
```

```
+-----+---+
| name|age|
+-----+---+
|  Bob| 35|
|Charlie| 40|
+-----+---+
```

```
+-----+-----+
|department |avg(age)|
+-----+-----+
|      HR |    40|
|Engineering|  37.5|
| Marketing|   28|
+-----+-----+
```

Observations and Inferences

- **Data Loading:** The program loads data efficiently into a DataFrame using Spark's built-in data reading capabilities.
- **Transformations:** The `filter` and `select` transformations were used to process and refine the data.
- **Aggregation:** The `groupBy` and `avg` functions in Spark allow easy grouping and aggregation operations, demonstrating Spark's ability to work with large datasets in parallel.
- **Performance:** Spark's distributed nature ensures that even with a large dataset, the operations can be run efficiently across multiple nodes, offering scalability.

Conclusion

In this simple Scala program using Apache Spark, we demonstrated how to:

1. Set up a Spark session and load data from a CSV file.
2. Perform basic data transformations like filtering and selecting specific columns.
3. Run aggregations to calculate the average age by department.

This simple example provides the foundational steps needed to start working with Apache Spark in a distributed environment. Spark's ability to process large-scale data efficiently is what makes it a go-to framework for big data processing. By building on this basic understanding, users can explore more advanced data transformations, machine learning algorithms, and real-time streaming operations in Spark.

Develop a Movie Recommendation Model Using the Scikit-learn Library in Python

Objective

The objective of this project is to develop a movie recommendation model using the scikit-learn library in Python. The model will use a dataset containing information about movies, such as movie genres, titles, and descriptions, to recommend similar movies to a user based on content similarity.

Hardware Requirements

- A computer or server with at least 4 GB of RAM (8 GB recommended for better performance with larger datasets).
- A minimum of 1 CPU core for basic data operations.
- An internet connection (to download the dataset and external dependencies).

Software Requirements

- **Python:** The programming language used for this task (version 3.x recommended).
- **scikit-learn:** A machine learning library that will be used for building the recommendation model.
- **pandas:** A library used for data manipulation and analysis.
- **numpy:** A library for numerical computations.
- **matplotlib** (optional): For data visualization.
- **jupyter notebook** or any Python IDE (e.g., PyCharm, Visual Studio Code) for running and testing the code.

To install the necessary packages, use the following:

```
pip install pandas numpy scikit-learn matplotlib
```

Theory

- **Recommender Systems:** A recommender system is a type of algorithm that provides personalized recommendations to users. These systems use data about users, items, and their interactions to suggest products, services, or content. Movie recommendation systems, for example, suggest movies based on user preferences, similar movies, or collaborative filtering.
- **Content-Based Filtering:** One of the common approaches in building recommendation systems. It recommends items (movies) similar to the ones the user has shown interest

in, based on item features such as genre, keywords, and descriptions.

- **TF-IDF Vectorization:** The Term Frequency-Inverse Document Frequency (TF-IDF) is a statistical method used to evaluate the importance of a word in a document relative to a corpus. In content-based recommendation, TF-IDF helps in converting the textual data (movie descriptions) into numerical data, which can then be used for similarity calculations.
- **Cosine Similarity:** A metric used to measure the cosine of the angle between two vectors in a multi-dimensional space. In the context of recommendation, it helps in calculating how similar two movies are based on their features.

Important Coding Concepts

1. **TF-IDF Vectorizer:** Converts text into a matrix of token counts adjusted by their importance (TF-IDF) to identify similarities between documents (in this case, movie descriptions).
2. **Cosine Similarity:** A metric used to measure the similarity between two vectors. In this case, it helps calculate how similar two movies are based on their descriptions.
3. **Pandas DataFrame:** A 2D data structure used to store data in rows and columns. Pandas will be used to load, manipulate, and explore the movie dataset.
4. **Scikit-learn:** A machine learning library that includes tools for vectorizing text, computing similarity metrics, and building recommendation models.
5. **Model Evaluation:** In a recommendation system, evaluating the accuracy of recommendations involves calculating how well the system can predict user preferences.

Code Implementation

Step 1: Import Libraries and Load Dataset

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Load the movie dataset
url = "https://github.com/rashida048/Some-NLP-Projects/raw/master/movie_dataset.csv"
df = pd.read_csv(url)
```

```
# Display the first few rows of the dataset  
df.head()
```

Step 2: Data Preprocessing

We will focus on the movie titles and descriptions for this recommendation system.

```
# Data Preprocessing  
# Clean any missing data in the 'overview' column (movie descriptions)  
df['overview'] = df['overview'].fillna("")  
  
# Extract relevant columns (Movie Title and Description)  
movies = df[['title', 'overview']]  
  
# Show the first few rows to ensure the data is loaded correctly  
movies.head()
```

Step 3: TF-IDF Vectorization

Convert the movie descriptions into a TF-IDF matrix.

```
# Initialize the TF-IDF Vectorizer  
tfidf = TfidfVectorizer(stop_words='english')  
  
# Transform the movie descriptions into a TF-IDF matrix  
tfidf_matrix = tfidf.fit_transform(movies['overview'])  
  
# Check the shape of the TF-IDF matrix  
print(tfidf_matrix.shape)
```

Step 4: Cosine Similarity Calculation

We will now compute the pairwise cosine similarity between all movies based on their descriptions.

```
# Calculate cosine similarity between the movies based on their TF-IDF vectors  
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)  
  
# Check the cosine similarity between the first movie and others  
cosine_sim[0]
```

Step 5: Build a Function for Movie Recommendations

Create a function that recommends movies based on the input movie title.

```
# Create a series with the movie titles as indices
indices = pd.Series(movies.index, index=movies['title']).drop_duplicates()

def get_recommendations(title, cosine_sim=cosine_sim):
    # Get the index of the movie that matches the title
    idx = indices[title]

    # Get the pairwise similarity scores of all movies with that movie
    sim_scores = list(enumerate(cosine_sim[idx]))

    # Sort the movies based on similarity scores
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

    # Get the indices of the top 10 most similar movies
    sim_scores = sim_scores[1:11] # Exclude the input movie itself

    # Get the movie indices
    movie_indices = [i[0] for i in sim_scores]

    # Return the top 10 most similar movies
    return movies['title'].iloc[movie_indices]

# Test the recommendation system with a movie title
get_recommendations("The Dark Knight")
```

Step 6: Display Recommendations

Once the function is executed, it will return a list of movie recommendations similar to the provided movie title.

Observations and Inferences

- **Data Preprocessing:** The dataset was cleaned, and missing values in the movie descriptions were handled by filling them with empty strings.
- **TF-IDF Vectorization:** By using the TF-IDF vectorizer, we transformed movie descriptions into a numerical format, allowing us to calculate their similarities.
- **Cosine Similarity:** We used cosine similarity to measure how similar movies are to one another based on their descriptions. The higher the cosine similarity, the more similar two movies are.
- **Movie Recommendations:** The recommendation function successfully returns a list of similar movies based on the input title, demonstrating the effectiveness of content-based filtering.

Conclusion

In this project, we developed a movie recommendation system using the scikit-learn library in Python. By leveraging the TF-IDF vectorization technique and cosine similarity, we were able to recommend movies based on their descriptions. This approach is a form of content-based filtering, where recommendations are made based on the attributes of the movies themselves, such as their genre and plot descriptions.

This model can be expanded and improved by integrating additional features like movie ratings, genres, or collaborative filtering for a more personalized recommendation system. This provides a foundation for building more complex recommendation systems and can be adapted for various domains beyond movies.

Covid Vaccine Analytics on State Wise Vaccination Data

Objective

The objective of this task is to perform various analytics on the "Covid Vaccine Statewise" dataset provided on Kaggle. We will analyze the vaccination data for India, including:

1. Describing the dataset to understand its structure.
2. Identifying the number of people vaccinated with the first and second doses across different states.
3. Analyzing the gender-wise vaccination data to identify the number of males and females vaccinated.

Hardware Requirements

- A computer or server with at least 4 GB of RAM (8 GB or more is recommended for large datasets).
- Minimum of 2 CPU cores for efficient data processing.
- A network connection to access the dataset from Kaggle (if not downloading locally).

Software Requirements

- **Python:** Programming language used for data analysis.
- **pandas:** A powerful Python library for data manipulation and analysis.
- **matplotlib / seaborn** (optional): For data visualization.
- **Jupyter Notebook** or any IDE like PyCharm or Visual Studio Code for running the code.

To install the necessary libraries, use the following:

```
pip install pandas matplotlib seaborn
```

Theory

- **COVID-19 Vaccination Data:** The dataset provides information on COVID-19 vaccinations administered across various states in India. The columns in the dataset include details about total vaccination numbers for both doses, male and female vaccination counts, and the status of vaccinations in each state.
- **Data Description:** A brief overview of the dataset allows us to understand its structure, columns, and data types, helping us to prepare for meaningful analysis.

- **Vaccination Analysis:**

- **First Dose:** The number of individuals who have received their first dose of the vaccine.
- **Second Dose:** The number of individuals who have received their second dose of the vaccine.
- **Gender Analysis:** Analyzing the number of male and female individuals vaccinated can provide insights into vaccination coverage across genders.
- **Data Cleaning and Preprocessing:** This involves checking for any missing data, handling any inconsistencies, and transforming the data into a format that can be easily analyzed.

Important Coding Concepts

1. **pandas DataFrame:** A 2D data structure that allows you to store and manipulate tabular data.
2. **Data Cleaning:** Handling missing or incorrect values, data formatting, and ensuring that the dataset is ready for analysis.
3. **Aggregation:** Grouping data by specific features (such as state or dose) and summarizing it to get relevant insights.
4. **Data Visualization:** Creating visualizations using libraries like **matplotlib** or **seaborn** to present the analysis results.
5. **Basic Statistical Functions:** Calculating totals, means, and other basic statistics for data analysis.

Code Implementation

Step 1: Import Libraries and Load Dataset

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset from a URL or local path
url = "https://www.kaggle.com/sudalairajkumar/covid19-inindia?select=covid_vaccine_statewise.csv"
df = pd.read_csv(url)

# Display the first few rows of the dataset
df.head()
```

Step 2: Describe the Dataset

The `describe()` function provides an overview of the dataset's structure, including column names, data types, and basic statistics.

```
# Describe the dataset to understand its structure
df.info()
```

```
# Display summary statistics of numeric columns
df.describe()
```

Step 3: Number of Persons Vaccinated for the First Dose by State

```
# Summing the total number of first doses administered across states
first_dose = df[['State', 'Total Doses Administered']].copy()
first_dose['First Dose'] = df['Total Doses Administered'] - df['Second Dose Administered']

# Display the number of persons vaccinated for the first dose in each state
first_dose = first_dose[['State', 'First Dose']].sort_values(by='First Dose', ascending=False)
print(first_dose)
```

Step 4: Number of Persons Vaccinated for the Second Dose by State

```
# Display the number of persons vaccinated for the second dose in each state
second_dose = df[['State', 'Second Dose Administered']]
second_dose = second_dose.sort_values(by='Second Dose Administered', ascending=False)
print(second_dose)
```

Step 5: Number of Males Vaccinated

```
# Calculate the number of males vaccinated across states
males_vaccinated = df[['State', 'Male Vaccinated']].copy()
males_vaccinated = males_vaccinated.sort_values(by='Male Vaccinated', ascending=False)
print(males_vaccinated)
```

Step 6: Number of Females Vaccinated

```
# Calculate the number of females vaccinated across states
females_vaccinated = df[['State', 'Female Vaccinated']].copy()
females_vaccinated = females_vaccinated.sort_values(by='Female Vaccinated',
ascending=False)
print(females_vaccinated)
```

Step 7: Data Visualization (Optional)

Visualize the number of people vaccinated for the first and second doses in different states using bar plots.

```
# Plotting the number of first doses administered in each state
plt.figure(figsize=(10, 6))
```

```
sns.barplot(x='State', y='First Dose', data=first_dose, palette='viridis')
plt.title('Number of First Doses Administered Across States')
plt.xticks(rotation=90)
plt.show()
```

```
# Plotting the number of second doses administered in each state
plt.figure(figsize=(10, 6))
sns.barplot(x='State', y='Second Dose Administered', data=second_dose, palette='viridis')
plt.title('Number of Second Doses Administered Across States')
plt.xticks(rotation=90)
plt.show()
```

Step 8: Gender-wise Vaccination Comparison (Optional)

```
# Plotting the number of male vs female vaccinations
plt.figure(figsize=(10, 6))
df_mf = df[['State', 'Male Vaccinated', 'Female Vaccinated']].copy()
df_mf.set_index('State').plot(kind='bar', stacked=True, figsize=(10, 6))
plt.title('Male vs Female Vaccination by State')
plt.ylabel('Number of People Vaccinated')
plt.xticks(rotation=90)
plt.show()
```

Observations and Inferences

- **Dataset Structure:** The dataset contains the state-wise COVID vaccination information, including the total doses administered, number of males and females vaccinated, and the second dose administered. Each row represents data for a specific state.
- **Vaccination by First Dose:** The number of people vaccinated for the first dose varies widely across states, with some states having higher totals due to population density or better vaccination drive management.
- **Vaccination by Second Dose:** Similarly, the data for the second dose indicates a significant portion of the population has completed the vaccination process, though it may be lower than the first dose in some states.
- **Gender-wise Analysis:** Vaccination counts for males and females are provided separately, helping to understand gender-wise participation in the vaccination process.
- **Visualization Insights:** The bar plots for first and second doses across states highlight regions where vaccination efforts have been successful or need further attention.

Conclusion

The analysis of the COVID vaccine statewise dataset provides useful insights into the vaccination progress across India. By examining the number of first and second doses administered, as well as the gender breakdown, we can identify patterns and regional disparities. The data visualizations offer an easy-to-understand summary of these findings, allowing policymakers to take targeted action in areas with lower vaccination rates.

This analysis can be further extended by incorporating demographic information, tracking vaccination rates over time, or comparing the data with COVID-19 case rates to understand the correlation between vaccination and infection control.
